

Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers

James O’Neill*
Fin AI Research

james.oneill@intercom.io

Fergal Reid
Fin AI Research

Abstract

Looped, weight-tied Transformers reduce parameters by reusing a single block, but decoding still stores a separate K/V cache for every recurrence step. We show that this loop-indexed cache is highly structured. For a fixed token, layer and head, K/V vectors trace a short low-rank trajectory across loops, while the head and layer axes remain much flatter. We introduce Looped Latent Attention (LLA), a post-training cache codec that stores compact K and V latents and reconstructs loop-specific K/V vectors only when attention reads them. The default per-head codec compresses recurrence, while LLA-2D also folds heads into one latent for the extreme-compression regime. The codec is initialized from the SVD of teacher activations and refined with logit and attention-output distillation. At matched cache budget, per-head LLA outperforms head-axis MLA, cross-layer sharing, KV quantization and final-loop reuse, showing that the recurrent cache is low-rank but not safely collapsible to a single state. The same axis advantage holds on Ouro-2.6B-Thinking and transfers to Huginn-3.5B, where an SVD codec remains near-lossless to $32\times$ compression in decoder-independent evaluation. The cache reduction is exact. On one H200, the latent-store path increases measured Ouro-1.4B batch capacity at 4k context from 32 to 768 sequences at $21.3\times$ compression. Lastly, for long reasoning rollouts such as in MATH-500, on-policy refinement on student-generated prefixes raises accuracy at $4\times$ compression from 0.43 to 0.66 and reduces no-answer generations when compared to token-level off-policy distillation.

1 Introduction

At inference, the memory that constrains a Transformer is its KV cache, not its weights. The weights are loaded once and shared across the whole batch. The cache is private to each request and grows with the context and with every generated token. Past a certain context length, this per-request state, not the parameter count, decides how many requests fit on a device and whether batched serving stays economical.

Looped, weight-tied Transformers attack the parameter side of this cost by applying one block repeatedly. Universal Transformers (Dehghani et al., 2019) recast depth as recurrence, and recent looped language models such as Ouro/LoopLM (Zhu et al., 2025) scale the idea to billions of parameters. However, the cache is untied. Each recurrence step recomputes attention and writes its own keys and values, so the cache is indexed by recurrence step in addition to layer, head, and token. A model with $T = 4$ loops over $D = 24$ layers therefore carries the KV cache of a 96-layer decoder while sharing the parameters of a 24-layer one.

This cache carries far fewer degrees of freedom than its nominal size. For a fixed token, layer, and head, the per-loop K/V vectors trace a short, low-rank trajectory, and the redundancy is specific to the loop axis: at the same budget, the head and layer spectra are much flatter. Looped Latent Attention (LLA) exploits this by replacing the T per-loop K/V vectors with single latent and loop-specific reconstruction maps.

*Corresponding author.

An important question is *which axis to compress and what the KV cache must preserve when it does*. A looped decoder invites a degenerate shortcut, if recurrence merely converges to a fixed point, one could store the final loop and reuse it everywhere. It also inherits the usual Transformer options of compressing heads, sharing layers, and quantizing values. Our experiments separate these by only changing what the cache stores while the teacher is kept frozen and the attention pattern and recurrence intact. Under this controlled swap, final-loop reuse buys the same $4\times$ reduction for free yet collapses GSM8K generation to zero, while a small low-rank trajectory code holds performance at matched-budget. In the following, we list our three primary contributions.

Contributions.

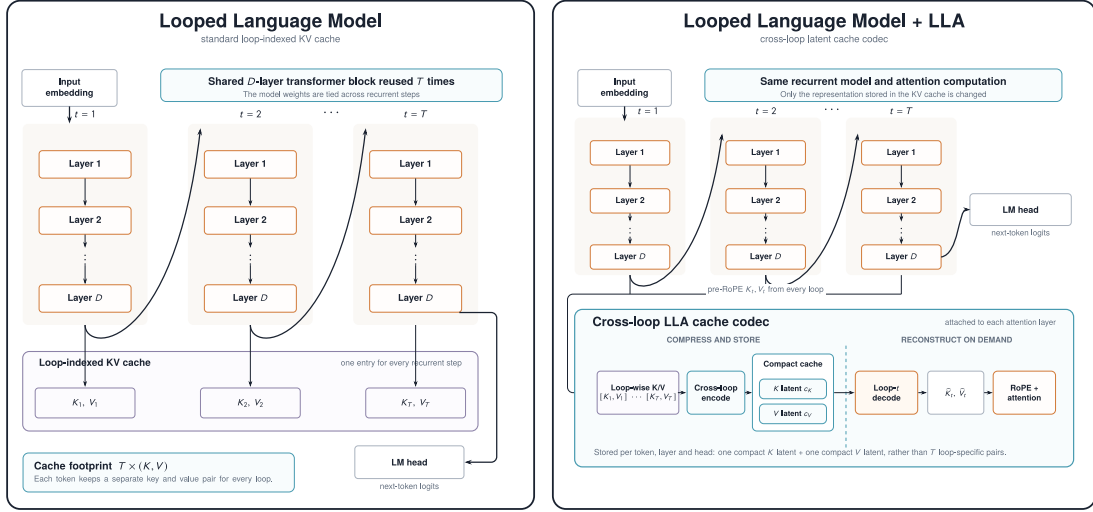
- LLA, a cross-loop latent K/V codec for weight-tied looped Transformers. It stores $(r_k + r_v)H$ scalars per token per layer instead of $2THd_{\text{head}}$, giving compression $\rho = 2Td_{\text{head}}/(r_k + r_v)$. LLA-2D extends the same family by folding the heads into one latent for the extreme-compression regime.
- We show that recurrence is the most compressible cache axis in Ouro-1.4B, Ouro-2.6B-Thinking and Huginn-3.5B. K and V follow different trajectories, motivating $r_v > r_k$, and matched-budget comparisons against head-axis MLA, cross-layer sharing, KV quantization and final-loop reuse favor the loop axis.
- We tie the results to deployment: an explicit on-policy refinement objective uses student-generated prefixes to stabilize long rollouts, and the exact scalar reduction translates into measured serving gains, raising Ouro-1.4B maximum batch from 32 to 768 sequences at $21.3\times$ compression.

2 Related Work

Looped and Universal Transformers. Universal Transformers (Dehghani et al., 2019) apply a weight-tied block for several steps, casting depth as recurrence. Recent looped language models carry this to billion-parameter LMs and trade computation for parameter efficiency on reasoning workloads (Zhu et al., 2025). Mechanistic studies of these models report that per-loop computation settles onto low-dimensional trajectories (Blayney et al., 2026), and LLA builds a cache codec on exactly that structure. Recurrence does not address serving cost, where attention still stores a separate K/V state for every loop.

Latent and compressed KV caches. Multi-head Latent Attention (MLA) introduced in DeepSeek-V2 compresses the standard Transformer KV cache by storing a low-rank latent across the head/channel axis and reconstructing per-head keys and values (DeepSeek-AI, 2024). Cross-layer sharing targets the layer axis (Brandon et al., 2024; Sun et al., 2024; Liu et al., 2024), quantization targets precision (Hooper et al., 2024), and token eviction targets the sequence axis (Zhang et al., 2023; Xiao et al., 2024). All of these operate on single-pass Transformers. LLA instead compresses recurrence, an axis that only weight-tied looped models have. Because the same block and projections are reused at every step, the loop axis is redundant for a reason the head and layer axes lack. The head axis, targeted by multi-query and grouped-query attention (Shazeer, 2019; Ainslie et al., 2023), composes on top of LLA (LLA-2D), and sequence-axis eviction multiplies with it (Sec. 7).

Memory-efficient looped Transformers. MELT (Vendrell et al., 2026) is the closest concurrent work. It distills a single gated K/V state per layer, while LLA keeps the loop trajectory and compresses its rank. However, as we will see, our zero-parameter final-loop-reuse control, which collapses the loop to one state, fails at the same cache budget. The two methods also span different regimes. A collapsed state fixes the memory reduction at roughly the loop count, whereas a tunable latent can sit below or above that point by choice of r_k and r_v .



Looped Latent Attention (LLA). A standard looped language model writes a separate key/value cache entry at every recurrent step. LLA inserts a lightweight codec only on the cache path: it compresses the trajectory across loops into separate K and V latents, then reconstructs the requested loop's $\hat{K}_t$ and $\hat{V}_t$ immediately before RoPE and attention. The recurrent block, model weights, and attention computation remain unchanged.

Figure 1: **Cross-loop Looped Latent Attention.** **Left:** A weight-tied looped language model reuses the same D -layer decoder stack for T recurrent steps. In the standard implementation, each step contributes a separate key-value pair (K_t, V_t) to a loop-indexed KV cache, so the stored cache grows with the number of recurrent steps. **Right:** Looped Latent Attention replaces these loop-specific cache entries with a cross-loop codec. For each layer, token, and attention head, the keys and values produced across the T recurrent steps are collected along the loop axis and compressed into separate compact K and V latents. At recurrent step t , loop-specific up-projections reconstruct only the K_t and V_t required by attention. Reconstruction occurs before RoPE and multi-head attention, leaving the recurrent computation, model weights, and attention operation unchanged; only the representation stored in the KV cache is modified.

3 Looped Latent Attention

Looped models save parameters but still materialize K/V states for every recurrence step. We compress this recurrence axis with a codec that retrofits onto a frozen teacher and changes only the representation held in memory, leaving the recurrent model and attention computation otherwise unchanged. As illustrated in Figure 1, the standard loop-indexed KV cache is replaced by compact cross-loop K and V latents, from which the cache entries required at each recurrence step are reconstructed on demand.

We first define the uncompressed recurrent cache, then introduce the cross-loop latent cache and derive its exact compression ratio. The same construction is applied separately to K and V, potentially with different ranks. We then describe the SVD initialization and the matched baselines used to test whether the loop axis is the appropriate dimension to compress.

Teacher cache. Consider a weight-tied looped Transformer with D layers, H attention heads of width d_{head} and T recurrence steps. For one layer, one head, one token and one axis (K or V), the teacher emits T vectors. We stack them as

$$x = [x_1; \dots; x_T] \in \mathbb{R}^{Td_{\text{head}}}. \quad (1)$$

The uncompressed cache stores both K and V for every loop, head and layer, for a per-token per-layer cost of $2THd_{\text{head}}$ scalars.

Cross-loop codec. For each layer, head and axis, LLA stores a centered low-rank latent

$$c = (x - \mu)W_{\text{down}}, \quad c \in \mathbb{R}^r, \quad (2)$$

where $\mu \in \mathbb{R}^{Td_{\text{head}}}$ is a calibration-set mean and $W_{\text{down}} \in \mathbb{R}^{Td_{\text{head}} \times r}$. Loop t is reconstructed by a loop-specific up-projection,

$$\hat{x}_t = cW_{\text{up},t}^\top + \mu_t, \quad W_{\text{up},t} \in \mathbb{R}^{d_{\text{head}} \times r}. \quad (3)$$

K and V have separate latents and ranks, r_k and r_v , with $r_v > r_k$ because the value cache stabilizes more slowly across loops than the key cache. The compressed cache stores $(r_k + r_v)H$ activation scalars per token per layer, so

$$\rho = \frac{2THd_{\text{head}}}{(r_k + r_v)H} = \frac{2Td_{\text{head}}}{r_k + r_v}. \quad (4)$$

The ratio counts the per-request cache. The fixed codec matrices are shared across tokens, contexts and batches and do not scale with sequence length.

RoPE and reconstruction. The latent is fit to the pre-RoPE content projection. In a looped decoder, a token has the same position at every recurrence step, so the rotary phase (Su et al., 2021) is shared across loops and can be applied after reconstruction. This is simpler than standard MLA, which requires an explicit decoupled-RoPE branch to preserve positional information along the head/channel axis.

The LLA-2D variant (loop $\times$ head). The per-head codec of Eq. (2) can be pushed further by folding the head axis into the same latent. LLA-2D stores a single latent per token and layer and reconstructs *all* H heads’ K/V from it, MLA-style, in addition to the per-loop reconstruction of Eq. (3). This lowers the cache to $(r_k + r_v)$ scalars per token per layer, a factor H below per-head LLA, at the cost of coupling heads through one shared code. LLA-2D is the extreme-compression member of the family. Its down-projection input is the full THd_{head} stack rather than Td_{head} , so its parameter count grows with head count and recurrence length. This becomes the binding constraint at large T (the Huginn transfer, Section 5). Section 5 characterizes the accuracy/compression trade between the two members. LLA without qualification denotes the per-head variant.

Initialization and conversion. For each layer, head and axis we collect teacher K/V activations on a calibration set, center by the per-loop mean and initialize W_{down} from the top- r right singular vectors of the stacked-loop matrix. The initial up-projections are the corresponding loop slices. The SVD initialization is then fine-tuned with the teacher frozen. The objective is the forward KL from the teacher distribution to the swapped model plus an attention-output matching term,

$$\mathcal{L} = \text{KL}(p_{\text{teacher}} \| p_{\text{swap}}) + \frac{\lambda_{\text{attn}}}{TD} \sum_{t,d} \|a_{t,d} - \hat{a}_{t,d}\|_2^2, \quad (5)$$

with $\lambda_{\text{attn}} = 0.5$ in the reported runs. Only codec parameters are updated. The SVD prior matters more than any other design choice. At $4\times$, SVD initialization reaches held-out KL 0.105 after the short conversion, while random initialization stays at 0.771 under the same budget (Table 10, Fig. 12).

On-policy refinement. Eq. (5) is teacher-forced, fitting the codec on prefixes visited by the teacher. For long autoregressive rollouts, the swapped model instead conditions on prefixes produced by its own codec. We therefore add an optional second stage after the off-policy fit. The swapped model samples completions, the frozen teacher scores those sampled prefixes, and only codec parameters are updated. No reward or correctness filtering is used. Section 6 writes the off-policy and on-policy objectives side by side. The on-policy version replaces teacher prefixes with student-generated prefixes and stops the gradient through sampling.

Matched-axis baselines. The main ablations place comparable latent codecs on other cache axes under the same scalar budget, testing whether the loop axis is the right one. The head-axis baseline is per-loop MLA (no cross-loop sharing). The layer baseline reuses K/V across adjacent layers. The precision baseline is low-bit KV quantization. We also include a zero-parameter control that caches only the final-loop K/V and reuses it for every loop. That control has the same $4\times$ cache footprint as LLA for $T = 4$ and directly tests whether the loop trajectory can be replaced by its endpoint.

4 Cross-loop structure in the teacher cache

Before comparing trained codecs, we ask whether the frozen teacher already contains the structure that LLA assumes. This is a stricter test than downstream accuracy, since it examines the cache tensor directly,

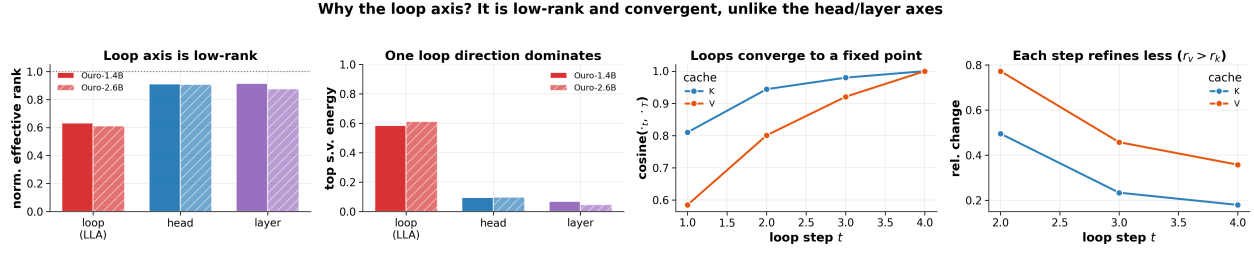


Figure 2: **Cross-loop K/V is low-rank, and its per-loop trajectory stabilizes.** Left two panels: the loop axis has much lower normalized effective rank than the head or layer axes, and a single direction carries a large fraction of the energy. Right two panels: K and V move toward the final loop while the step-to-step change shrinks, and V converges more slowly than K, motivating $r_v > r_k$.

without optimization, decoding heuristics or task-specific effects. If the loop axis is not spectrally different from the head or layer axes, then a cross-loop codec would be an arbitrary design choice. If it is lower-rank in the teacher, the later iso-cache ordering has a mechanistic explanation.

The loop-axis advantage is visible before training any codec. Fig. 2 compares the singular spectra of frozen-teacher K/V along the loop, head and layer axes. In Ouro-1.4B and Ouro-2.6B-Thinking, the loop axis has normalized effective rank about 0.61 to 0.63, with a single direction carrying roughly 60 percent of the energy. The head and layer axes are close to full-rank by the same measure. This explains why a small loop latent preserves more downstream behavior per cached scalar than head or layer compression. Stabilization across loops is a related but distinct property. A contraction can converge while staying full-rank, so we read the low rank directly from these spectra rather than infer it from convergence.

Cross-loop dynamics also explain the K/V rank split. Across recurrence steps, K and V move toward the final loop while step-to-step changes shrink. K stabilizes earlier, while V moves slower and carries more cross-loop variance. We therefore allocate $r_v > r_k$ throughout. The matched-budget ablation confirms the benefit, with $r_k/r_v = 96/160$ outperforming a shared-rank 128/128 codec at the same scalar count (Table 10).

A single endpoint is not enough. Fig. 3 shows that each token-head unit is dominated by one cross-loop direction, but the mass does not sit on the rank-1 line. Only 6 percent of K variance and 17 percent of V variance lie on the loop axis, and most variation remains content variation. The right object is therefore a short low-rank trajectory, not a collapsed state. This distinction is reinforced by the final-loop reuse baseline in Table 1.

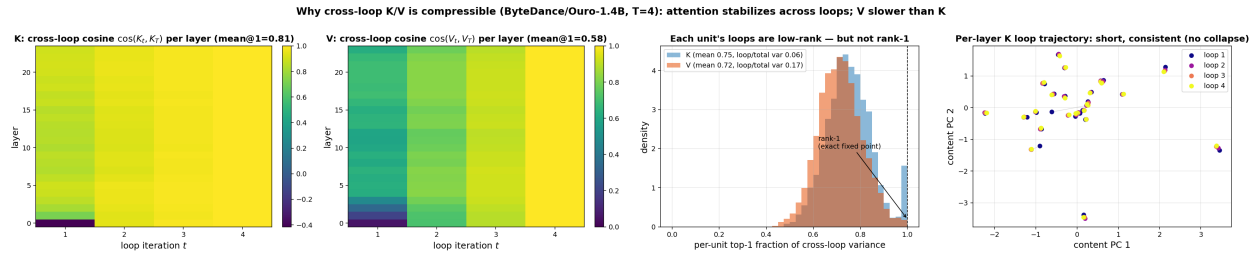


Figure 3: **Loop trajectories are low-rank, not collapsed.** In frozen Ouro-1.4B, cross-loop K/V cosines increase toward the final loop, and one direction captures most but not all cross-loop variance. The PCA view shows short, consistent paths rather than a single shared endpoint.

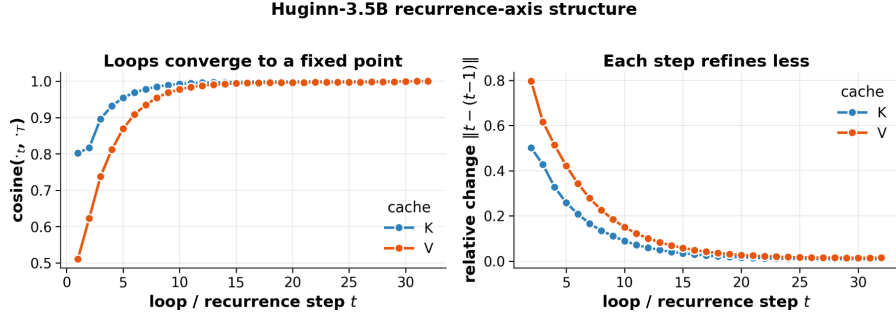


Figure 4: **The recurrence-axis mechanism transfers to Huginn-3.5B.** These are the same two convergence diagnostics as the right two panels of Fig. 2, now applied to a different family. Huginn has up to 32 recurrence steps and a different looped-LM architecture, yet the pattern is unchanged. Successive steps converge toward a fixed point, each step refines less, and V converges more slowly than K. Reproducing the identical diagnostic at 32 steps, far beyond Ouro’s 4, is stronger evidence for this mechanism.

The same mechanism appears in a second looped family. Huginn-3.5B uses a different recurrent-depth architecture and up to 32 recurrence steps (Geiping et al., 2025). Its loop axis is far more redundant than its head axis, and its K/V states converge toward a fixed point across recurrence steps (Fig. 4). The value cache again converges more slowly than the key cache. This recurrence-axis structure supports treating LLA as a property of weight-tied iteration rather than an Ouro-specific artifact. The remaining experiments test that structure through downstream accuracy, matched-budget baselines, long rollouts and serving capacity.

5 Experiments

The core question is which axis to spend rank on. At a fixed cache budget, we test whether compressing the loop beats compressing heads, layers or precision, and how much rank a converted codec needs before it matches the teacher on a generative task. Two further experiments ask whether the answer holds at a larger Ouro model and in the unrelated Huginn family. Table 8 is the cold-start rank sweep, Table 7 the broad-suite teacher comparison, Table 1 the matched-budget axis comparison, and Table 10 the design ablations. The reported ratios are representative operating points spanning the light, middle and extreme regimes.

Setup. Unless stated otherwise, generative results use the cached reconstruction decoder, which inserts reconstructed K/V before attention. The latent-store decoder in Sec. 7 is the memory-realizing path. Decoder-independent metrics evaluate the swapped model on teacher-provided tokens and are not affected by generation drift. The main teacher is Ouro-1.4B with $T = 4$ loops. Scale experiments use Ouro-2.6B-Thinking, and family-transfer experiments use Huginn-3.5B.

Quality under compression. Figure 5 summarizes Ouro-1.4B quality as the KV-cache is compressed. Across the broad suite (a), effects are task-dependent: code (HumanEval/MBPP) and MMLU-Pro stay at or above the teacher through $4\times$, while the reasoning tasks degrade, gently within the light-compression region ($\leq 2\times$) for GSM8K and BBH, and throughout for MATH-500, the most sensitive task (treated separately in Section 6, where long generations expose a different failure mode from teacher-forced fidelity). The GSM8K cold-start Pareto (b) sharpens the picture: strict-EM accuracy is flat near 0.58 from $4\times$ to $2.67\times$, then breaks sharply upward at $2\times$ to 0.75 and reaches 0.795 at $1.33\times$ (within the teacher’s sampling uncertainty), while the codec train-KL collapses over the same transition. This is a rank effect rather than a method ceiling: below the break the latent is large enough to preserve the decision tokens that determine final-answer correctness. The suite therefore separates two notions that are easy to conflate: per-token reconstruction can be strong while autoregressive rollout still needs a stability correction. Full numbers are in Appendix C.

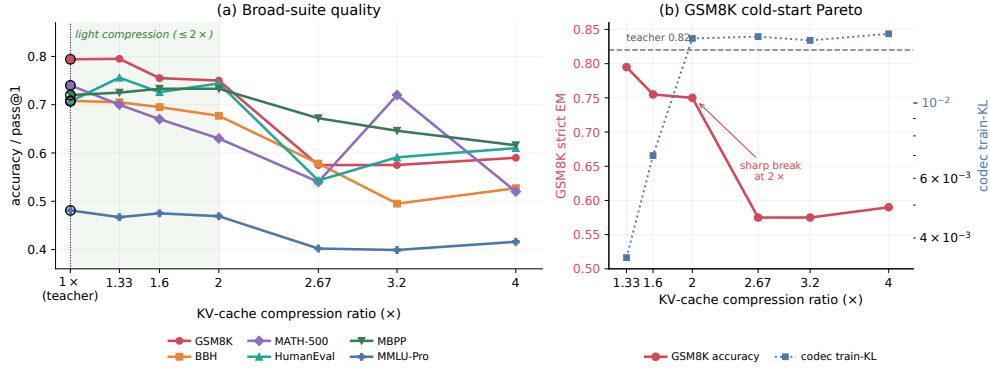


Figure 5: **Broad-suite quality against the Ouro-1.4B teacher as KV-cache compression increases.** Effects are task-dependent: code (HumanEval/MBPP) and MMLU-Pro stay at or above the teacher through $4\times$, while the reasoning tasks degrade - only gently within the light-compression region ($\leq 2\times$, shaded) for GSM8K and BBH, but throughout for the more sensitive MATH-500, and sharply beyond $2\times$ for GSM8K and BBH. GSM8K is 5-shot strict EM HumanEval/MBPP are EvalPlus pass@1 (base); MATH-500 uses symbolic final-answer verification; BBH and MMLU-Pro use exact match. Full numbers are in Table 7.

Table 1: **Iso-cache comparison across compression axes.**

Method	ρ	train-KL $\downarrow$	GSM8K $\uparrow$	MC-avg $\uparrow$	MMLU $\uparrow$
teacher	$1\times$	n/a	0.794	0.646	0.682
final-loop reuse	$4\times$	n/a	0.000	0.443	0.537
LLA (loop)	$4\times$	0.059	0.800	0.640	0.672
mha_head (head)	$4\times$	0.122	0.522	0.585	0.657
cla (layer)	$4\times$	0.267	0.660	0.597	0.666
kv_quant (precision)	$4\times$	n/a	0.702	0.645	0.672
LLA (loop)	$5.33\times$	0.062	0.752	0.640	0.674
mha_head	$5.33\times$	0.123	0.180	0.576	0.640
LLA-2D (loop $\times$ head)	$5.33\times$	0.078	0.760	0.628	0.674
kv_quant (precision)	$5.33\times$	n/a	0.122	0.621	0.607
LLA (loop)	$21.33\times$	0.128	0.740	0.622	0.668
mha_head	$21.33\times$	0.128	0.680	0.613	0.633
LLA-2D (loop $\times$ head)	$21.33\times$	0.097	0.723	0.624	0.670

Iso-cache axis comparison. Table 1 compares loop compression against matched-budget alternatives. We find that compressing the loop axis (LLA, the highlighted rows) preserves GSM8K, while the head, layer and precision baselines and zero-parameter final-loop reuse do not. All learned codecs are matched by KV-cache budget. The LLA rows use the checkpoint selected under the iso-cache protocol, whereas Table 8 is a separate cold-start rank sweep. LLA (per-head) compresses the loop axis. LLA-2D (loop $\times$ head) is a family variant that additionally folds the head axis (all heads from one latent). The head, layer and precision rows are baselines that compress orthogonal axes. The zero-parameter final-loop reuse control has the same $4\times$ cache budget as LLA, but collapses generation (GSM8K 0.000). Its reconstruction relative error is 0.35 for K and 0.60 for V. Precision quantization (kv_quant) is inference-only (no codec training, hence no train-KL) and cannot exceed $\sim 8\times$. It is competitive at $4\times$ but collapses on GSM8K at $5.33\times$. Table 8 should be read separately as a cold-start rank sweep, whereas Table 1 uses the checkpoint and evaluation protocol adopted for the baseline comparison. At $4\times$, LLA has substantially lower train-KL than head or layer compression and much higher GSM8K accuracy. The zero-parameter final-loop reuse baseline has the same $4\times$ cache budget and no trainable parameters, but collapses GSM8K to 0.000. This is the most direct evidence that the loop trajectory cannot be replaced by its endpoint.

Within the LLA family, LLA-2D (loop $\times$ head) is competitive at high compression on teacher-forced metrics but trails the per-head variant on generation at $21.3\times$. This matches the spectral picture. LLA-2D spends part of its budget on the head axis, which is flatter and more expensive to compress. The two variants

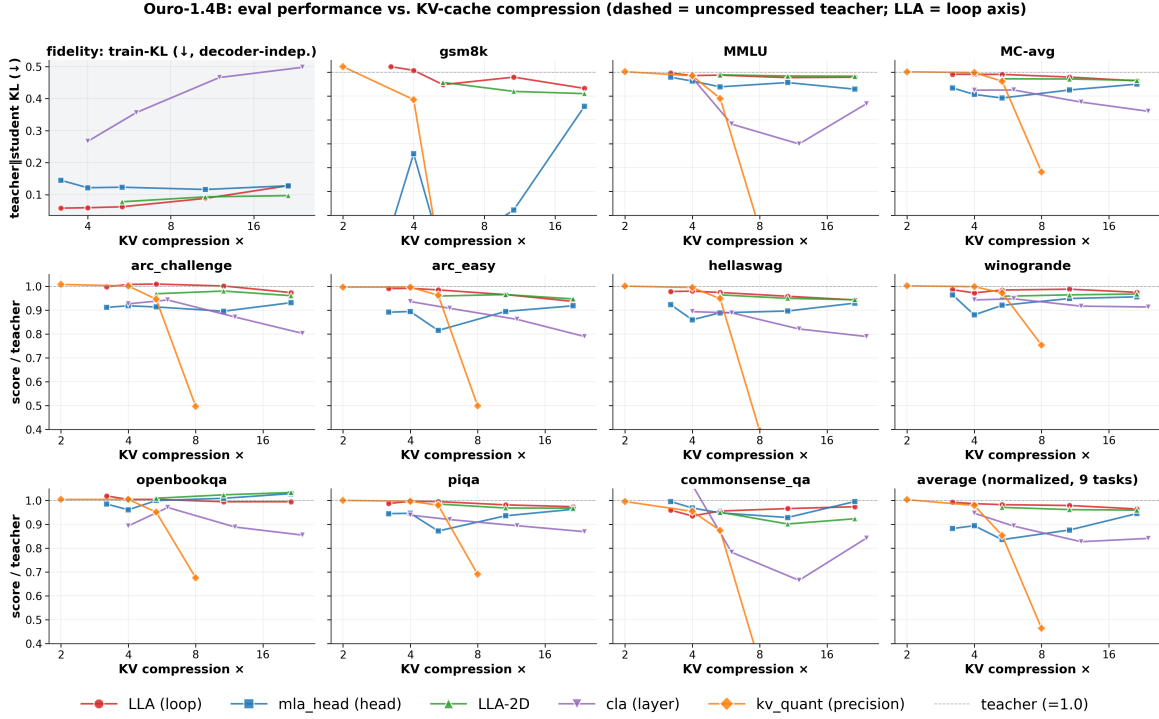


Figure 6: **Ouro-1.4B performance versus KV-cache compression.** Each panel plots an evaluation metric against compression for per-head LLA, the LLA-2D (loop $\times$ head) variant, and the head, layer and precision baselines. The dashed line is the uncompressed teacher. LLA tracks the teacher most closely across the suite, while precision quantization collapses on GSM8K past moderate compression.

therefore span the operating range, with per-head LLA for accuracy-preserving compression and LLA-2D for the extreme-compression regime where its $H \times$ smaller cache outweighs the accuracy gap. Table 1 reports the matched-budget numbers method by method, while Fig. 6 plots the same comparison as a frontier, where the loop-axis advantage is easiest to read. Per-head LLA does not win by overfitting a single benchmark, and the precision baseline is harmless at mild compression but brittle once the bit budget becomes too small.

Table 2: **Scale generalization on Ouro-2.6B-Thinking.** GSM8K is evaluated with the cached reconstruction decoder, while train-KL is decoder-independent. LLA remains near-lossless across 4 to 10.67 $\times$, while the head and layer baselines lose substantially more accuracy.

ρ	LLA (loop)	mla_head	LLA-2D	kv_quant
GSM8K EM $\uparrow$; teacher ≈ 0.83				
4 $\times$	0.834	0.585	0.66	0.814
5.33 $\times$	0.817	0.772	0.74	0.351
10.67 $\times$	0.821	0.775	0.82	n/a
train-KL $\downarrow$				
4 $\times$	0.057	0.150	0.192	n/a
5.33 $\times$	0.056	0.155	0.165	n/a
10.67 $\times$	0.070	0.121	0.110	n/a

Scale generalization. The 2.6B results in Table 2 preserve the ordering. LLA remains near-lossless on full GSM8K from 4 $\times$ to 10.67 $\times$, while the head baseline loses up to 0.25 absolute accuracy at 4 $\times$. Fig. 9 and Appendix G show that the advantage also holds across commonsense and knowledge tasks. The 2.6B model is deeper and has a stronger reasoning profile, and preserving the ordering under that change supports the structural interpretation from Section 4.

Design ablations. Table 10 isolates the choices used in the codec. SVD initialization is the largest factor, reducing held-out KL by an order of magnitude relative to random initialization under the same

conversion budget. The asymmetric rank split improves over a shared-rank latent at matched scalar cost, and attention-output matching gives a small additional gain beyond token KL. These ablations indicate that LLA mostly exploits teacher geometry rather than learning a new attention mechanism from scratch.

Transfer to Huginn. A LLA codec also works on Huginn-3.5B, where the recurrence length is much larger. Table 3 shows that SVD initialization alone is near-lossless to $32\times$ in decoder-independent fidelity. KL distillation recovers most of the $64\times$ and $128\times$ range. The downstream MC average is unchanged at $16\times$ and $32\times$ and degrades gracefully after that. On Huginn, head-axis SVD reconstruction is 20 to 30 times worse than LLA at matched budget, and LLA-2D (loop $\times$ head) becomes parameter-infeasible at this recurrence length because its down-projection input dimension grows with head count and loop count. Per-head LLA avoids this cost, which is why it is the default.

This transfer is intentionally harder than reproducing a number on a larger Ouro checkpoint. Huginn changes the recurrent-depth architecture, increases the realized recurrence to 32 steps, and is evaluated through decoder-independent fidelity rather than the Ouro cached generation path. The result indicates that a simple linear subspace found from teacher activations already captures most of the recurrence trajectory. Distillation is needed only when the requested budget asks the latent to represent the tail of the trajectory at $64\times$ or $128\times$. This is the behavior expected if weight-tied iteration itself is the source of the low rank, not a peculiarity of one training run or benchmark.

Table 3: **LLA on Huginn-3.5B with $R = 32$ recurrence steps.** Decoder-independent fidelity is measured against the uncompressed model on held-out text. A training-free SVD codec is near-lossless to $32\times$, and short KL distillation recovers the 64 to $128\times$ range. MC-avg is a broad downstream loglikelihood suite.

ρ	r_k/r_v	SVD KL $\downarrow$	SVD top-1	distill KL $\downarrow$	best top-1	MC-avg
teacher	n/a	n/a	n/a	n/a	n/a	0.523
$16\times$	128/256	0.005	0.970	n/a	0.970	0.523
$32\times$	64/128	0.010	0.954	n/a	0.954	0.525
$64\times$	32/64	0.197	0.799	0.145	0.834	0.486
$128\times$	16/32	0.763	0.620	0.177	0.812	0.473

6 Long-generation stability

Teacher-forced conversion trains the codec on teacher trajectories. At inference, however, the swapped model conditions on tokens produced by its own codec. A small early reconstruction error can move later prefixes away from the conversion distribution, and the error then compounds autoregressively. This failure is most visible on long MATH-500 generations, where the model can produce long non-terminating solutions even when teacher-prefix KL is low.

Fig. 7 bins $3.2\times$ completions by generated length, with the uncompressed teacher as a reference. Off-policy accuracy degrades with length far below the teacher and the no-answer rate spikes in the longest bin. On-policy distillation trains on the codec’s own sampled rollouts and closely tracks the teacher across length bins. The teacher’s own gentle decline with length reflects intrinsic problem difficulty, not exposure bias.

The refinement objective makes the distribution shift explicit. Let p_T denote the frozen teacher and p_S^θ the swapped model with codec parameters θ :

$$\mathcal{L}_{\text{off}}(\theta) = \mathbb{E}_{y \sim p_T} \left[\frac{1}{|y|} \sum_t \text{KL} (p_T(\cdot | y_{<t}) \| p_S^\theta(\cdot | y_{<t})) \right], \quad (6)$$

$$\mathcal{L}_{\text{on}}(\theta) = \mathbb{E}_{\tilde{y} \sim \text{sg}(p_S^\theta)} \left[\frac{1}{|\tilde{y}|} \sum_t \text{KL} (p_T(\cdot | \tilde{y}_{<t}) \| p_S^\theta(\cdot | \tilde{y}_{<t})) \right]. \quad (7)$$

Off-policy distillation uses teacher prefixes y . On-policy distillation instead uses student prefixes $\tilde{y}$ sampled from the current compressed model. The stop-gradient blocks gradients through sampling, so updates still come only from the KL term. The teacher remains the target distribution in both objectives. The on-policy

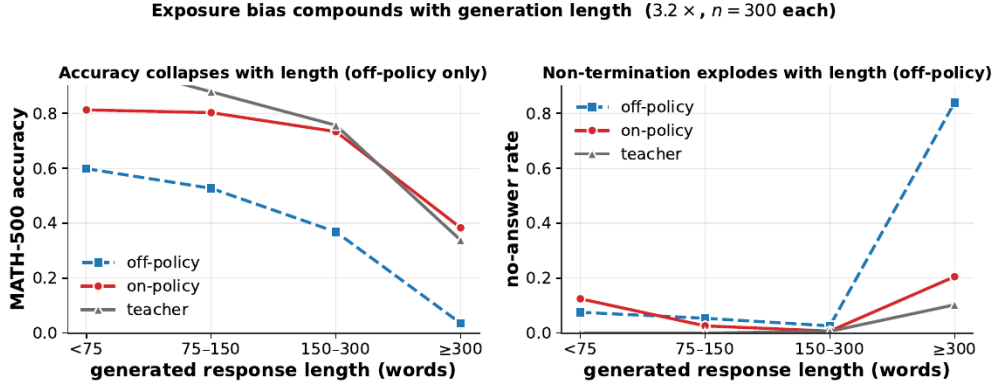


Figure 7: **Off-policy failure compounds with generation length.** At $3.2\times$, the uncompressed teacher (gray) sets the reference. Accuracy declines gently with length from problem difficulty, and non-termination stays low. On-policy distillation (red) tracks the teacher closely at every length, whereas the off-policy codec (blue) loses accuracy as responses grow longer and its no-answer rate spikes in the longest bin. The gap between off-policy and the teacher is the exposure-bias failure that on-policy training removes.

stage only changes which prefixes are used to tune the codec, turning rollout drift into a reconstruction problem on states the deployed model actually visits.

Table 4: **On-policy distillation across the rank Pareto.** MATH-500 is scored by reliable final-answer extraction plus symbolic verification (teacher 0.74). On-policy training improves long-generation stability at every ratio and gives the largest gains at 3.2 to $5.33\times$. “no-ans” denotes no extractable final answer.

ρ	off-policy	on-policy	Δ	off no-ans	on no-ans
teacher ($1\times$)	0.740	n/a	n/a	0.020	n/a
$3.2\times$	0.460	0.700	+0.24	0.21	0.06
$4\times$	0.430	0.660	+0.23	0.15	0.03
$5.33\times$	0.530	0.690	+0.16	0.12	0.03
$10.67\times$	0.650	0.690	+0.04	0.09	0.07
$21.33\times$	0.660	0.620	-0.04	0.11	0.05

Table 4 shows the full rank Pareto. On-policy distillation raises MATH-500 accuracy by 0.16 to 0.24 at 3.2 to $5.33\times$ and reduces no-answer generations at every ratio. At $21.3\times$, it still improves termination but does not improve accuracy, indicating that the smallest latent has already lost task-relevant information. Larger re-evaluations confirm the low-compression lift in Fig. 11. For long rollouts, the conversion data must include states visited by the compressed model itself.

Memory is only useful if accuracy survives at length. Figure 8 probes exact passkey retrieval (a 5-digit key at random depth, up to 64k) and reveals a clean *compressibility-length* trade-off: the reliable-retrieval length (longest context with retrieval ≥ 0.9) shrinks steeply as compression rises: $\geq 64k$ at $\leq 5.33\times$, 16k at $10.67\times$, 8k at $14.2\times$. Two failure modes bound the operating range. In a *length-limited* regime ($10.67\text{--}14.2\times$) retrieval is perfect up to a knee and then decays gracefully, as the fixed latent budget is spread over more positions and the farthest keys are lost first (the knee marches $16k\rightarrow 8k$). In a *compression-limited* regime ($\geq 15.5\times$) retrieval is flat and below 1.0 at *every* length, because the latent is too small to preserve the exact key-token identity at all. The trade-off also shifts with model *size*: at $10.67\times$ the 2.6B model retains perfect retrieval to 32k, where the 1.4B model has already fallen to 0.80 (Fig. 8, right), consistent with a larger model carrying more redundant KV and thus tolerating the same compression to longer context. Full numbers are in Appendix I; the two-pass prefill that reaches 64k is described in Appendix J (the 2.6B 64k cell OOMs on that prefill, a memory limit rather than an accuracy failure).

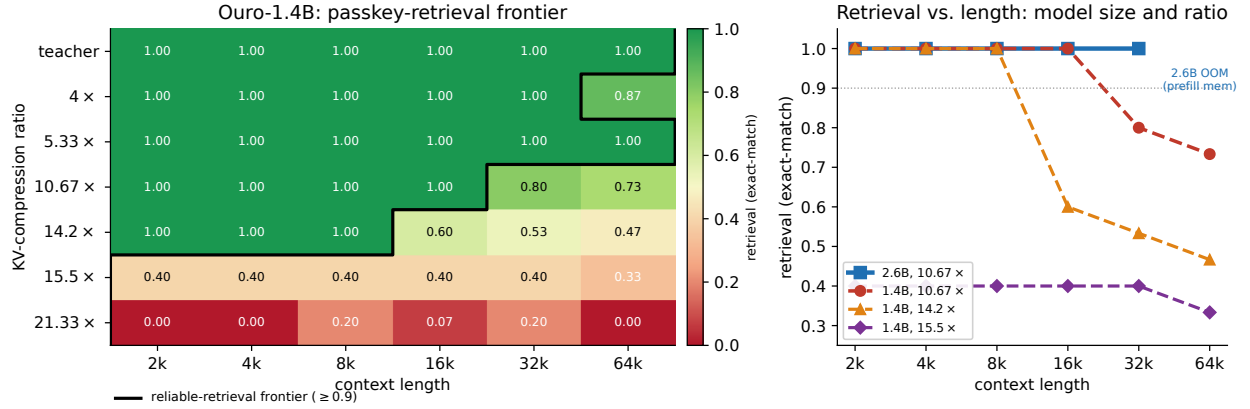


Figure 8: **Passkey retrieval under KV-cache compression.** *Left:* exact retrieval for Ouro-1.4B over the (compression ratio $\times$ context length) grid (15 trials/cell; every cell is a measured run, full numbers in Table 12). The black staircase is the empirical reliable-retrieval frontier (the ≥ 0.9 iso-retrieval contour through measured cells). *Right:* retrieval vs. length for the 1.4B model at 10.67/14.2/15.5 $\times$ and the 2.6B model at 10.67 $\times$, exposing both degradation regimes and the model-size effect in one view.

7 Serving and compute

Reducing per-request cache increases the contexts and batches that fit in memory (Kwon et al., 2023). For LLA, this benefit is exact on memory and implementation-dependent on compute: Eq. (4) reduces the cache by ρ , while runtime depends on how attention consumes the latent.

Table 5: **Memory, capacity and latency.** LLA reduces cached scalars exactly by ρ . At fixed VRAM and context length 4096, measured maximum batch follows the expected capacity gain. The full-memory fast path caches reconstructed K/V and is fast, while the latent-store capacity path realizes the memory saving and is reconstruction-compute-bound in the current implementation. Combining 21.3 $\times$ with token eviction reduces the 1M-context 1.4B cache further to 36 MiB. “n/a” marks an operating point not characterized for that path rather than a missing measurement. The fast path is profiled at its intended 4 $\times$ point (it caches full K/V and does not realize the memory saving at higher ρ), the capacity path at 21.3 $\times$, and the 2.6B capacity test was run to 4 $\times$.

Quantity	teacher	LLA 4 $\times$	LLA 21.3 $\times$
KV / token, 1.4B / 2.6B	768 / 1536 KiB	192 / 384 KiB	36 / 72 KiB
KV at 1M context, 1.4B	768 GiB	192 GiB	36 GiB
max batch at 4k, 1.4B	32	128	768
max batch at 4k, 2.6B	16	64	n/a
decode latency, fast path	63.6 ms/tok	74.1 ms/tok	n/a
peak decode throughput, capacity path	748 tok/s	n/a	108 tok/s

Table 5 and Fig. 10 translate the scalar reduction into capacity. At context length 4096 on one H200, the measured maximum batch for Ouro-1.4B rises from 32 to 128 at 4 $\times$ and 768 at 21.3 $\times$; Ouro-2.6B-Thinking rises from 16 to 64 at 4 $\times$. The tests fabricate the decode cache, push batch to OOM, and validate latent attention against full reconstruction.

There are two implementation regimes. A full-memory fast path caches reconstructed K/V and runs at 74.1 versus 63.6 ms/token (1.16 $\times$ teacher latency), but does not realize the memory reduction. The latent-store path caches only the latent and realizes the $\rho\times$ saving; because the current kernel reconstructs cached tokens on read, it is compute-bound (108 versus 748 peak tokens/s at the reported endpoints). The present systems result is therefore a capacity gain, not a decode-speed claim.

LLA also composes with sequence-axis eviction, and moderate compression preserves exact passkey retrieval at long context (Sec. 6; Tables 9 and 12). Per-head LLA favors fidelity, whereas LLA-2D trades accuracy for

a smaller single latent. An absorbed-attention formulation could avoid explicit reconstruction, but post-hoc decoupled RoPE remains less faithful than the full-RoPE reconstruction used for the main results. Appendix J gives the eviction, long-prefill, variant, and absorbed-attention details.

8 Discussion

The evidence supports recurrence-specific rather than axis-agnostic cache redundancy. Reapplying the same block and K/V projections to a stabilizing recurrent state produces a much steeper loop spectrum than the head or layer spectra; matched-budget codecs on those axes lose more accuracy, and the ordering transfers from Ouro to Huginn. At the same time, final-loop reuse collapses generation at the same $4\times$ budget. Recurrence may approach a fixed point, but its intermediate cache states remain functionally distinct: the compressible object is a low-rank trajectory, not a single endpoint.

Rank defines the deployment regime. On Ouro-1.4B, $1.33\text{--}2\times$ is close to a drop-in replacement across the broad suite. At roughly $3\text{--}5\times$, teacher-forced fidelity remains useful, but long rollouts benefit from on-policy refinement because the compressed model visits its own prefixes. Beyond $10\times$, LLA is primarily a capacity tool: many likelihood tasks remain usable, while exact retrieval and long-form reasoning become increasingly rank-limited. The $21.3\times$ result, where on-policy training improves termination but not accuracy, separates correctable rollout drift from irreversible information loss.

The recurrence axis is orthogonal to head sharing, cross-layer reuse, quantization, and token eviction. For looped decoders, the spectral results suggest compressing recurrence first and stacking another method only when the deployment target justifies its additional quality cost. This priority is a consequence of weight-tied iteration and need not hold for a conventional single-pass Transformer.

Our central claims, exact cache reduction and accuracy under full-RoPE reconstruction, are established independently of any throughput speedup. We nevertheless implement and evaluate an absorbed-attention compute path. Because the attention score is bilinear, the key up-projection can be moved to the query, allowing attention to read the latents directly. In a shape-matched random-weight decode benchmark that isolates kernel geometry from trained-model accuracy, this path is $2.3\times$ faster than reconstruct-then-attend at 262k context and matches teacher decode latency with $4\times$ less cache. Its post-training fidelity is bounded by a decoupled-RoPE floor, which we characterize rather than leave open: a learned query adapter lowers KL from 0.55 to 0.34, whereas full-RoPE reconstruction reaches 0.059, because the frozen query was trained against full-RoPE keys. This localizes the dominant mismatch and identifies native training, aligning query projections with the latent cache during pretraining, as the route to closing it and converting memory savings into compute savings. A complementary direction is to exploit low-rank residual updates across loops. The measurements are consistent with this premise because the loop trajectory is low-rank without collapsing to its endpoint, but they establish low rank in the cache rather than in the full recurrent computation. Appendix J gives the implementation and complete decoupled-RoPE characterization.

9 Conclusion

Looped Transformers tie weights but not their recurrence-indexed KV cache. LLA replaces that cache with a low-rank cross-loop K/V trajectory, outperforms matched-budget head, layer, precision, and endpoint baselines, and transfers across two looped-model families. Its exact scalar reduction yields measured serving-capacity gains. These results make recurrence a practical compression axis for looped decoders and a natural target for native latent-cache training.

More broadly, the result reframes what weight tying costs at inference. Tying parameters across loops makes recurrent-depth models cheap to store, but leaves their KV cache untied and growing with the loop count, so at long context and large batch it is serving cost, not parameter count, that limits looped reasoning models. The same tied computation that shares parameters also drives the per-loop states onto a low-rank trajectory, so the axis responsible for the cost is also the most compressible one. Treating recurrence as a first-class compression axis therefore converts a serving liability into a design lever. We expect the same structure, a tied computation producing an untied but low-rank cache, to recur in other iterative or weight-shared architectures, where a cache indexed by an internal iteration could be compressed by the analogous construction.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- Hugh Blayney, Alvaro Arroyo, Johan Obando-Ceron, Pablo Samuel Castro, Aaron Courville, Michael M. Bronstein, and Xiaowen Dong. A mechanistic analysis of looped reasoning language models, 2026. URL <https://arxiv.org/abs/2604.11791>.
- William Brandon et al. Reducing transformer key-value cache size with cross-layer attention. *arXiv preprint arXiv:2405.12981*, 2024.
- DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024. Introduces Multi-head Latent Attention (MLA).
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.
- Coleman Hooper et al. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- Akide Liu et al. Minicache: Kv cache compression in depth dimension for large language models. *arXiv preprint arXiv:2405.14366*, 2024.
- Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Yutao Sun et al. You only cache once: Decoder-decoder architectures for language models. *arXiv preprint arXiv:2405.05254*, 2024.
- Victor Conchello Vendrell, Arnau Padres Masdemont, Niccolo Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. Memory-efficient looped transformer: Decoupling compute from memory in looped language models. *arXiv preprint arXiv:2605.07721*, 2026.
- Guangxuan Xiao et al. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
- Zhenyu Zhang et al. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025. Ouro/LoopLM; models ByteDance/Ouro-1.4B, Ouro-2.6B-Thinking.

A Experimental details

Models. All experiments use frozen, publicly released looped/recurrent-depth Transformers. **Ouro-1.4B** (the main teacher): $D = 24$ weight-tied layers, $H = 16$ attention heads of width $d_{\text{head}} = 128$, run at $T = 4$ recurrence steps. **Ouro-2.6B-Thinking**: deeper Ouro variant, also $T = 4$, used for the scale study. **Huginn-3.5B**: a different recurrent-depth family, evaluated at $R = 32$ recurrence steps for the family-transfer study. The teacher weights are never updated; only the K/V codec is trained.

LLA codec and conversion. For each layer, head and axis (K, V) we collect the teacher’s pre-RoPE per-loop activations on a fixed calibration set of 255 prompts and initialize W_{down} from the top- r right singular vectors of the stacked-loop matrix (loop slices give the initial $W_{\text{up},t}$). The SVD-initialized codec is then fine-tuned with the teacher frozen using `train_rmla.py`: objective = forward KL(teacher || swapped) + $\lambda_{\text{attn}} = 0.5$ per-loop attention-output matching (Eq. 5); 3000 steps (300 for the design ablations of Table 10), batch size 8, sequence length 512, AdamW, learning rate $1e-3$ with 100-step warmup and cosine decay to 5%, held-out KL evaluated every 200 steps ($n = 32$). We select the *best-KL* checkpoint (not the final step); a codec whose final-step KL exceeds its best is reported at its best checkpoint. Ranks (r_k, r_v) per compression ρ are listed in Table 8; we use $r_v > r_k$ throughout (V converges more slowly, Sec. 4). **LLA-2D** shares the same recipe but stores one latent per token and layer, reconstructing all H heads from it. RoPE is applied *after* reconstruction (the rotary phase is shared across loops for a token), so no decoupled-RoPE branch is needed for the content path.

Matched-axis baselines. All learned baselines use the identical conversion recipe above; only the codec architecture changes. *mha_head*: per-loop MLA (head-axis latent, no cross-loop sharing). *cla*: cross-layer K/V sharing over $s \in \{4, 6, 12, 24\}$ adjacent layers. *kv_quant*: inference-only low-bit K/V quantization at $\{2, 3, 4, 8\}$ bits (no codec training, hence no train-KL, and it cannot exceed $\sim 8\times$). *final-loop reuse*: a zero-parameter control that caches only the last loop’s K/V and reuses it for all loops.

Evaluation suite and protocols. **GSM8K**: 5-shot at 1.4B and 2-shot at 2.6B, strict-match EM, `max_gen_toks=256`. The 1.4B teacher is scored on the full 1319-item test set; the iso-cache and broad-suite codecs use a 100-item subset with the faithful re-forward decoder, while the tier4 cold-start sweep (Table 8) uses a 200-item subset with the earlier streaming reconstruction decoder and a 100-item teacher reference. At 2.6B, scored with the faithful decoder, the teacher and the per-head and head-axis arms use the full 1319 items and the LLA-2D arms use a 100-item subset. **MATH-500**: 4-shot, `max_gen_toks=2048`, batch 1, faithful decoder, scored by `math_verify` symbolic equivalence; teacher on all 500 items, codecs on 200. **BBH**: 3-shot, `max_gen_toks=1024`, batch 1, exact match, faithful decoder; teacher 25 items/sub-task (675 docs) and codecs 5/sub-task (135 docs, identical items across arms). **HumanEval/MBPP**: EvalPlus pass@1 (base/plus) on the full sets (164/378). **MMLU-Pro**: exact match on the full set. The commonsense/knowledge MC suite (**ARC-e/c**, **HellaSwag**, **PIQA**, **WinoGrande**, **OpenBookQA**, **CommonsenseQA**, **MMLU**) is 0-shot loglikelihood accuracy on the full sets and is decoder-independent. Within each table the codec and baseline arms are scored on identical items; the teacher reference is scored on the full set where the two differ. Passkey (Table 12) hides a 5-digit key at a random depth in filler context, 15 trials/cell (5 keys $\times$ 3 depths), scored by exact retrieval; contexts 2k to 64k. “train-KL” is the held-out teacher-forced KL of the codec and is decoder-independent.

Decoding and caching. Two decoders are used. The *cached reconstruction decoder* is two-pass: pass 1 captures the teacher’s pre-RoPE per-loop K/V, the codec reconstructs them, and pass 2 re-forwards forcing the reconstruction so the model’s own RoPE and cache store the post-RoPE reconstruction. This matches the training objective and is used for the iso-cache, broad-suite, 2.6B and MATH-500/BBH numbers; the tier4 cold-start sweep used an earlier streaming reconstruction decoder. The *capacity / latent-store decoder* caches only the latent and reconstructs on read, realizing the $\rho\times$ memory saving; its two-pass prefill is streamed layer-by-layer so long prompts (to 64k) fit in memory. Two serving regimes (Sec. 7): a full-memory *fast path* (caches reconstructed full K/V, teacher-latency, no memory saving) and a *capacity path* (stores latents, reconstruction-compute-bound). On-policy refinement samples the codec’s own rollouts (temperature

sampling, no reward/correctness filtering) and minimizes the frozen teacher’s per-token KL to those visited states with a stop-gradient through sampling.

Hardware and precision. Codecs are trained and evaluated in bfloat16. Serving-capacity and latency measurements (Table 5) are on a single H200 at context length 4096; generative evaluations use B200 GPUs (183 GB) with per-process memory fraction 0.99 for the long-context and 2.6B runs.

Broad-suite accuracy relative to the teacher. Table 6 restates the broad suite (Table 7) as compressed/teacher ratios, putting every task on one scale so the task-dependence of compression is directly comparable.

Table 6: **Broad-suite accuracy relative to the Ouro-1.4B teacher** (compressed/teacher, computed from Table 7). A value of 1.00 matches the teacher. The common scale makes the task-dependence explicit. Code (HumanEval, MBPP) and MMLU-Pro retain the most under compression, while GSM8K and BBH degrade fastest.

Model	GSM8K	HE b	HE p	MBPP b	MBPP p	MATH-500	BBH	MMLU-Pro
Ouro teacher (1×)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
LLA 1.33×	1.00	1.07	1.06	1.01	1.02	0.95	1.00	0.97
LLA 1.60×	0.95	1.03	1.03	1.02	1.05	0.91	0.98	0.99
LLA 2.00×	0.94	1.05	1.04	1.02	1.01	0.85	0.96	0.98
LLA 2.67×	0.72	0.77	0.73	0.93	0.95	0.73	0.82	0.84
LLA 3.20×	0.72	0.84	0.80	0.90	0.90	0.97	0.70	0.83
LLA 4.00×	0.74	0.86	0.84	0.86	0.86	0.70	0.74	0.86

B Additional figures

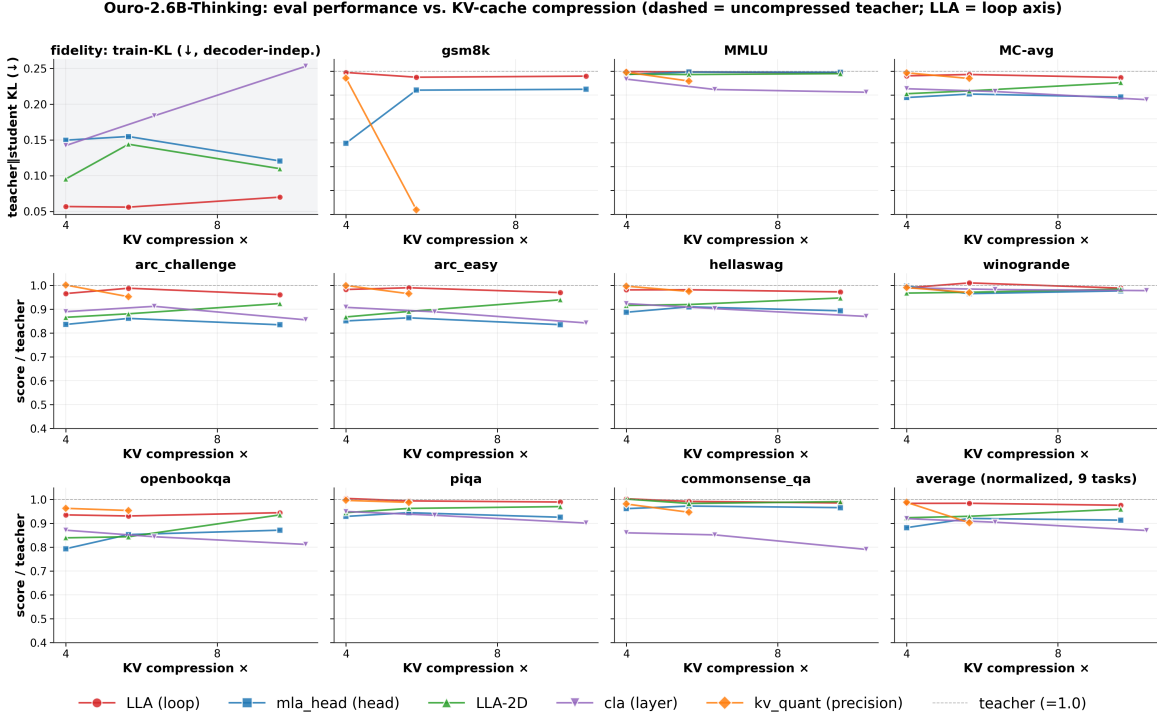


Figure 9: **Ouro-2.6B-Thinking performance versus KV-cache compression.** The layout matches Fig. 6. The loop-axis codec remains closest to the teacher on train-KL, GSM8K and the commonsense/knowledge suite. The precision baseline cannot reach the largest ratios without collapse.

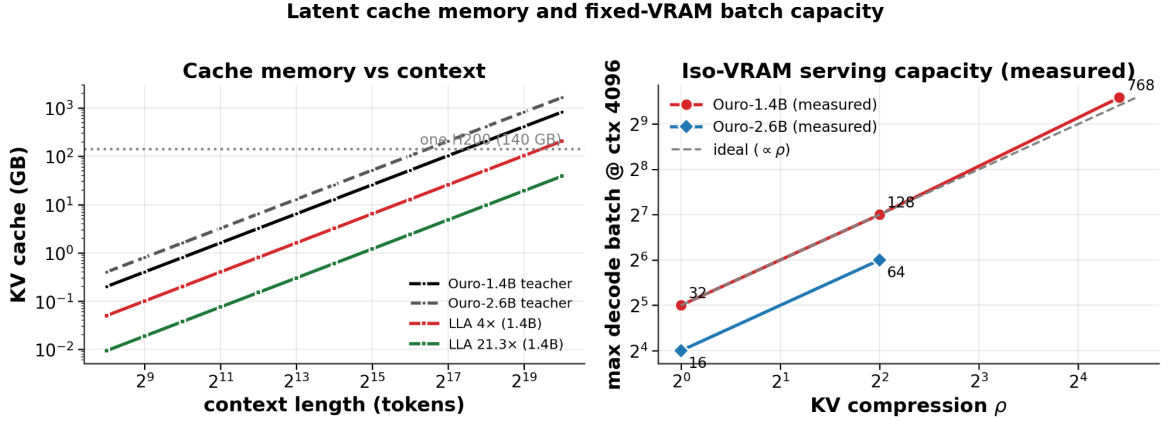


Figure 10: **The latent cache gives an exact memory reduction and measured serving capacity.** Left: cache memory scales linearly with context and is divided by the compression ratio. Right: at fixed VRAM and context length 4096, maximum batch size follows the ideal ρ scaling in measured capacity tests.

C Compression Across The Broad Suite

Table 7: **Quality against the Ouro-1.4B teacher across the broad suite.** GSM8K is 5-shot strict EM; HumanEval and MBPP use EvalPlus pass@1 (base/plus); BBH and MMLU-Pro use exact match. MATH-500 uses reliable final-answer extraction and symbolic verification. Light compression matches the teacher across tasks, while heavier compression is task-dependent.

Model	GSM8K	HumanEval b/p	MBPP b/p	MATH-500	BBH	MMLU-Pro
Ouro teacher (1 $\times$)	0.794	0.707 / 0.671	0.720 / 0.595	0.74	0.708	0.481
LLA 1.33 $\times$	0.795	0.756 / 0.713	0.725 / 0.606	0.70	0.705	0.467
LLA 1.60 $\times$	0.755	0.726 / 0.689	0.733 / 0.624	0.67	0.695	0.475
LLA 2.00 $\times$	0.750	0.744 / 0.695	0.733 / 0.603	0.63	0.677	0.469
LLA 2.67 $\times$	0.575	0.543 / 0.488	0.672 / 0.566	0.54	0.578	0.402
LLA 3.20 $\times$	0.575	0.591 / 0.537	0.646 / 0.537	0.72	0.495	0.399
LLA 4.00 $\times$	0.590	0.610 / 0.561	0.616 / 0.513	0.52	0.527	0.416

Across the broad suite (Table 7), light compression matches or exceeds the teacher within evaluation noise. Code and BBH retain non-trivial performance at 4 $\times$, but lighter ratios are needed to match the teacher. MATH-500 is the most sensitive task and is treated separately in Section 6, because long generations stress rollout stability rather than only per-token reconstruction. Appendix Table 6 restates these numbers as accuracy relative to the teacher, on a common scale.

The broad-suite table also separates three regimes. Light compression (1.33 to 2 $\times$) is effectively a drop-in replacement for the full cache, resolving enough of the loop trajectory that residual differences fall within benchmark variation. By 2.67 to 5.33 $\times$, the representation still carries many likelihood and short-generation tasks, though long reasoning becomes sensitive to prefix drift. At the most aggressive ratios the codec is a capacity operating point, not a universal quality-preserving conversion. The matched-axis comparisons below use the middle and high-compression regimes to test whether recurrence is the right axis to spend the latent dimension on.

The regimes reflect one underlying variable, generation length. Robustness to compression falls off as generations grow longer. Multiple-choice and short-generation tasks tolerate the middle regime with little loss, code and BBH sit in between, and long-form mathematical reasoning breaks first. A deployment should budget around this ordering rather than any single benchmark.

Table 8: **GSM8K accuracy-compression Pareto for Ouro-1.4B.** Six cold-start LLA codecs are trained with the same recipe and evaluated with the cached reconstruction decoder. Each multiplier is the exact KV-cache compression ratio ρ (Eq. (4)). The plateau from $4.0\times$ to $2.67\times$ breaks sharply at $2.0\times$.

Model	(r_k, r_v)	GSM8K strict $\uparrow$	codec train KL $\downarrow$
teacher	n/a	0.820	n/a
LLA $4.00\times$	(96, 160)	0.590	0.0160
LLA $3.20\times$	(120, 200)	0.575	0.0153
LLA $2.67\times$	(144, 240)	0.575	0.0157
LLA $2.00\times$	(192, 320)	0.750	0.0155
LLA $1.60\times$	(240, 400)	0.755	0.0070
LLA $1.33\times$	(288, 480)	0.795	0.0035

Accuracy-compression frontier. Table 8 shows a cold-start GSM8K Pareto sweep for Ouro-1.4B. Accuracy is flat around 0.58 from $4\times$ to $2.67\times$, then rises sharply at $2\times$ and reaches 0.795 at $1.33\times$, close to the teacher. The result is a rank effect rather than a method ceiling. Below the transition, the latent is large enough to preserve the decision tokens that determine final-answer correctness. The sharp break shows where the loop trajectory becomes sufficiently resolved for GSM8K generation.

D Token-eviction composition table

Table 9: **LLA composes with token eviction at no extra quality cost.** GSM8K (5-shot, strict EM) under StreamingLLM/H2O-style eviction to a 260-token budget (sink 4 + window 256). The $\sim 1\text{k}$ -token 5-shot prompts are $\sim 74\%$ evicted, so the sequence-axis cost is real (both rows fall from ≈ 0.79 without eviction; teacher/LLA- $4\times$ no-eviction reference in Table 1). Eviction on a near-lossless 8-bit codec isolates the sequence axis, and LLA- $4\times$ adds the recurrence axis on top. The two remain close, confirming that the axes are orthogonal. LLA reconstructs the *kept* tokens accurately and adds little beyond eviction’s own cost. Cache size composes exactly (Table 5). LLA- $21.3\times$ + eviction caps the 1M-context 1.4B cache at 36 MiB.

Method (260-token eviction budget)	GSM8K strict $\uparrow$
eviction alone (kv_quant-8, sequence axis)	0.510
LLA $4\times$ + eviction (loop $\times$ sequence)	0.520

E On-policy rank Pareto (figure)

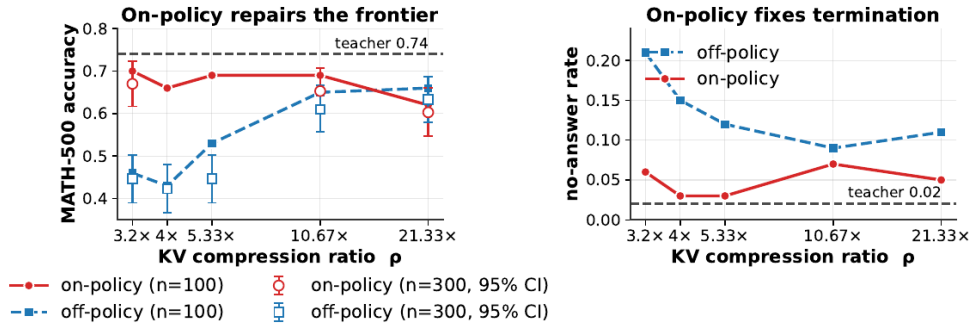


Figure 11: **On-policy distillation stabilizes the long-generation Pareto.** On reliably scored MATH-500, self-rollout distillation raises the low-compression points and reduces no-answer generations throughout the Pareto. Hollow markers show independent re-evaluations with bootstrap 95% confidence intervals.

F Design-ablation detail

Table 10: **Design-choice ablations.** Held-out teacher-forced KL for a $4\times$ codec after a matched 300-step conversion. The reference is SVD initialization, split ranks $r_k/r_v = 96/160$ and attention-output matching.

Ablation	Variant	held-out KL $\downarrow$
init prior	SVD (ours)	0.105
init prior	$t = 1$ SVD	0.128
init prior	random	0.771
rank split	$r_k < r_v$ (96/160)	0.105
rank split	shared rank (128/128)	0.129
loss	KL + $\lambda_{\text{attn}} = 0.5$	0.105
loss	KL only	0.110

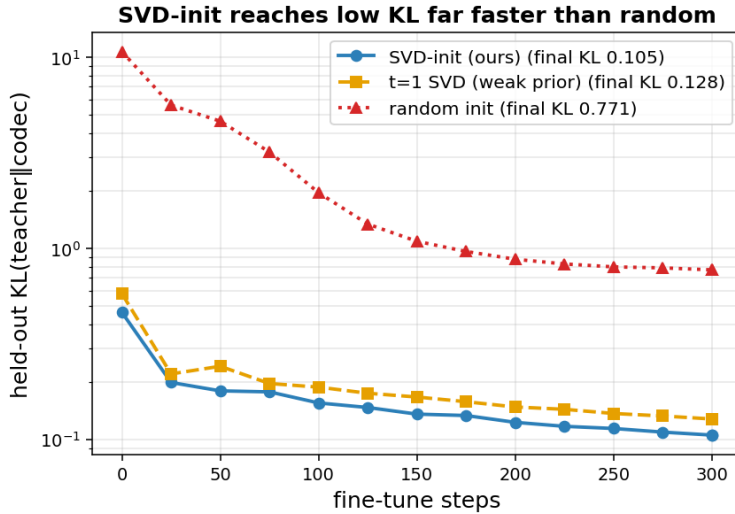


Figure 12: **SVD initialization dominates random initialization.** Held-out KL for the $4\times$ codec falls quickly from the SVD teacher prior. A random codec does not match SVD initialization within the same conversion budget.

G Full 2.6B commonsense and knowledge suite

Table 11: **Ouro-2.6B-Thinking: full commonsense and knowledge suite.** All tasks are 0-shot loglikelihood accuracy and decoder-independent. Per-head LLA is near-lossless at every budget and remains several points above the head and layer baselines and the LLA-2D variant.

Method	ρ	ARC-e	ARC-c	HSwag	WGr	C-QA	MMLU	PIQA	OBQA	Avg
teacher	1×	0.833	0.541	0.568	0.684	0.752	0.750	0.785	0.310	0.653
LLA	4×	0.818	0.521	0.557	0.676	0.754	0.749	0.782	0.302	0.645
LLA	5.33×	0.818	0.526	0.557	0.691	0.746	0.748	0.783	0.300	0.646
LLA	10.67×	0.813	0.526	0.552	0.676	0.741	0.745	0.780	0.288	0.640
mha_head	4×	0.733	0.460	0.494	0.681	0.723	0.740	0.729	0.234	0.599
mha_head	5.33×	0.751	0.474	0.504	0.660	0.731	0.747	0.739	0.244	0.606
mha_head	10.67×	0.730	0.461	0.498	0.668	0.726	0.747	0.725	0.256	0.601
LLA-2D	4×	0.738	0.463	0.513	0.661	0.753	0.741	0.742	0.242	0.607
LLA-2D	5.33×	0.753	0.477	0.520	0.664	0.740	0.740	0.760	0.236	0.611
LLA-2D	10.67×	0.786	0.512	0.539	0.672	0.745	0.742	0.761	0.294	0.631
cla	4×	0.756	0.484	0.518	0.676	0.647	0.725	0.750	0.260	0.602
cla	6×	0.769	0.493	0.505	0.672	0.640	0.692	0.739	0.254	0.596
cla	12×	0.732	0.468	0.484	0.669	0.595	0.684	0.717	0.250	0.575
kv_quant	4×	0.832	0.530	0.565	0.677	0.738	0.747	0.782	0.292	0.645
kv_quant	5.33×	0.816	0.519	0.550	0.663	0.712	0.719	0.777	0.292	0.631

H Cross-loop stabilization at 2.6B

Fig. 13 repeats the trajectory diagnostic on Ouro-2.6B-Thinking. The attention K/V stabilizes within the first loop, each token-head unit has a dominant but not exhaustive loop direction, and the value cache remains the higher-variance axis.

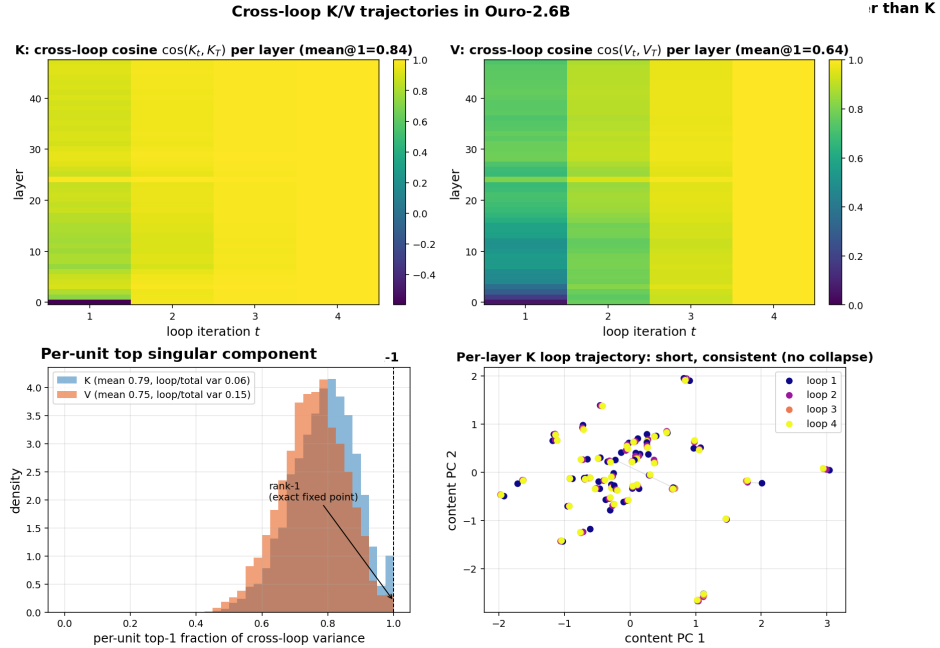


Figure 13: **Cross-loop K/V stabilization on Ouro-2.6B-Thinking.** The 2.6B model reproduces the same pattern as Ouro-1.4B. K/V stabilizes across loops, one direction dominates but does not explain all variance, and the value cache has higher cross-loop variance than the key cache.

I Long Context Retrieval Details

Table 12: **Long-context passkey retrieval under LLA compression (Ouro-1.4B)**. Exact-match retrieval of a 5-digit key hidden at a random depth in filler context. The uncompressed teacher retrieves perfectly to 64k. LLA is near-lossless for retrieval through 5.33 $\times$ across the whole range, with only the 4 $\times$, 64k cell dipping to 0.87. The 10.67 $\times$ point is the knee (perfect to 16k, degrading at 32 to 64k), while 21.33 $\times$ loses the exact token identity a key requires. LLA rows use the capacity/latent cached decoder, whose two-pass prefill reaches 64k via streamed layer-wise reconstruction (Sec. 7).

context	teacher	LLA 4 $\times$	LLA 5.33 $\times$	LLA 10.67 $\times$	LLA 21.33 $\times$
2,048	1.00	1.00	1.00	1.00	0.00
4,096	1.00	1.00	1.00	1.00	0.00
8,192	1.00	1.00	1.00	1.00	0.20
16,384	1.00	1.00	1.00	1.00	0.07
32,768	1.00	1.00	1.00	0.80	0.20
65,536	1.00	0.87	1.00	0.73	0.00

J Additional serving and discussion details

Eviction and long-context prefill. The recurrence and sequence axes compose multiplicatively. Under a 260-token StreamingLLM/H2O-style budget, eviction alone and eviction stacked on LLA-4 $\times$ score 0.51 and 0.52 on GSM8K (Table 9), so the codec adds little beyond eviction’s own quality cost. At the most aggressive reported point, LLA-21.3 $\times$ combined with a 1024-token eviction budget caps the 1M-context Ouro-1.4B cache at 36 MiB. Reaching the 64k passkey cells required reconstructing prefill K/V layer-by-layer and freeing each layer’s captures immediately; a one-shot implementation retains float32 captures and full-width reconstructions and exhausts memory beyond approximately 32k.

Per-head LLA versus LLA-2D at inference. Per-head LLA stores H latents per token and layer, $(r_k + r_v)H$ scalars, and reconstructs each head independently. LLA-2D stores one latent, $(r_k + r_v)$ scalars, and reconstructs all heads jointly. It therefore has a factor- H smaller cache for fixed ranks, whereas per-head LLA is more accurate at matched cache budget (Table 1). Both variants can use either the full-memory fast path or the latent-store capacity path; the choice between them is an operating-point decision between fidelity and maximum capacity.

Absorbed attention. The bilinear score permits the K up-projection to move to the query side, $q^\top (W_{\text{up}}c) = (W_{\text{up}}^\top q)^\top c$, while the V path satisfies $\sum_i a_i W_{\text{up}}^V c_i = W_{\text{up}}^V \sum_i a_i c_i$. Attention can therefore operate directly on latents. RoPE prevents exact absorption of the positional component, so the implementation uses an MLA-style NoPE content path plus a small explicit rotary branch (DeepSeek-AI, 2024); the rotary branch is shared across recurrence steps for each token. In a shape-matched random-weight decode benchmark at the 4 $\times$ geometry, which isolates the kernel dimensions from trained-model accuracy, the absorbed path is 2.3 $\times$ faster than reconstruct-then-attend at 262k context and reaches teacher decode latency with 4 $\times$ less cache (Table 13). The advantage grows with context because reconstruct-then-attend must materialize the full-width K/V at every step.

Table 13: **Absorbed decoupled decode latency** (Ouro-1.4B, 4 $\times$ geometry; shape-matched benchmark that isolates kernel dimensions from trained-model accuracy). As context grows, the absorbed path’s advantage over reconstruct-then-attend rises to 2.3 $\times$ and its per-token latency reaches teacher parity.

context L	teacher (ms/tok)	absorbed (ms/tok)	vs. reconstruct-then-attend	vs. teacher
32,768	0.66	1.19	1.22 $\times$	0.56 $\times$
131,072	2.29	2.32	2.17 $\times$	0.99 $\times$
262,144	4.46	4.25	2.30 $\times$	1.05 $\times$

For a frozen pretrained model, post-hoc decoupled RoPE introduces a fidelity floor because the query projection was trained against full-RoPE keys. Without adapting the query, the decoupled path has reconstruction KL 0.55; a learned query adapter lowers it to 0.34, and extended training reaches 0.30, while full-RoPE

reconstruction reaches 0.059 (Table 14). The adapter gain implicates query-key positional mismatch as the dominant limiting factor, but the remaining gap means that all main accuracy results use full-RoPE reconstruction. Native training can remove this mismatch by aligning the query projections with the latent cache from pretraining onward.

Table 14: **Post-hoc decoupled-RoPE characterization** (Ouro-1.4B, $4\times$, $d_{\text{rope}} = 64$). A learned query adapter lowers reconstruction KL and raises top-1 token agreement (to 0.84, and 0.88 with extended training), versus ~ 0.95 for full-RoPE. A sweep over $d_{\text{rope}} \in \{32, 64, 96\}$ preserves the ordering (smaller d_{rope} is worse, e.g. KL 1.04 at $d_{\text{rope}} = 32$ with the adapter). No post-hoc configuration reaches full-RoPE reconstruction, which is why all main accuracy results use the full-RoPE path.

configuration	reconstruction KL $\downarrow$
full-RoPE reconstruction (reference)	0.059
decoupled-RoPE, no query adapter	0.55
decoupled-RoPE + query adapter	0.34
decoupled-RoPE + query adapter, extended training	0.30

Native recurrent architectures and scope. A model trained natively with a decoupled latent cache could expose the latent directly to attention and align its query projections with the absorbed implementation. A more ambitious design could exploit low-rank residual updates across loops, computing shared content once and adding inexpensive loop-specific deltas. These are future architecture directions, not assumptions behind the reported accuracy or capacity numbers: the evaluated reconstruction path preserves the teacher’s attention computation and changes only the representation stored in the cache.